

Đồ Thị Nâng Cao

BỒI DƯỠNG KIẾN THỨC NỀN CP

Đỗ Phan Thuận



Email/Facebook: thuandp.sinhvien@gmail.com

Khoa Công nghệ thông tin
Trường Đại học Tôn Đức Thắng.

Ngày 25 tháng 10 năm 2025

- 1 Đồ thị có trọng số
- 2 Cấu trúc dữ liệu các tập không giao nhau – UNION-FIND
- 3 Cây khung nhỏ nhất - MST
- 4 Đường đi ngắn nhất
- 5 Một số bài toán kinh điển trên đồ thị
- 6 Một số đồ thị đặc biệt

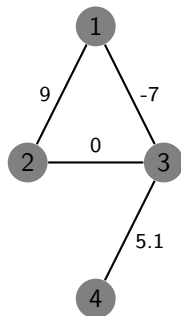
- 1 Đồ thị có trọng số
- 2 Cấu trúc dữ liệu các tập không giao nhau – UNION-FIND
- 3 Cây khung nhỏ nhất - MST
- 4 Đường đi ngắn nhất
- 5 Một số bài toán kinh điển trên đồ thị
- 6 Một số đồ thị đặc biệt

Đồ thị có trọng số

- Trong phần này ta xét đồ thị mà mỗi cạnh của nó có trọng số đi kèm
 - ▶ giá trị trên cạnh
 - ▶ trọng số trên cạnh
 - ▶ ví dụ: khoảng cách của con đường, chi phí truyền thông giữa hai nút mạng, ...
- Biểu diễn đồ thị có trọng số bởi danh sách kề mở rộng

Đồ thị có trọng số

```
struct edge {  
    int u, v;  
    int weight;  
  
    edge(int _u, int _v, int _w) {  
        u = _u;  
        v = _v;  
        weight = _w;  
    }  
};
```



Đồ thị có trọng số

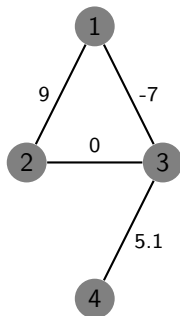
```
vector<edge> Adj[5];
```

```
Adj[1].push_back(edge(1, 2, 9));  
Adj[1].push_back(edge(1, 3, -7));
```

```
Adj[2].push_back(edge(2, 1, 9));  
Adj[2].push_back(edge(2, 3, 0));
```

```
Adj[3].push_back(edge(3, 1, -7));  
Adj[3].push_back(edge(3, 2, 0));  
Adj[3].push_back(edge(3, 4, 5.1));
```

```
Adj[4].push_back(edge(4, 3, 5.1));
```



- 1 Đồ thị có trọng số
- 2 Cấu trúc dữ liệu các tập không giao nhau – UNION-FIND
- 3 Cây khung nhỏ nhất - MST
- 4 Đường đi ngắn nhất
- 5 Một số bài toán kinh điển trên đồ thị
- 6 Một số đồ thị đặc biệt

Union-Find

- Cho n phần tử
- Cần quản lý vào các tập không giao nhau
- Mỗi phần tử ở trong đúng 1 tập
- $Items = \{1, 2, 3, 4, 5, 6\}$
- $Collections = \{1, 4\}, \{3, 5, 6\}, \{2\}$
- $Collections = \{1\}, \{2\}, \{3\}, \{4\}, \{5\}, \{6\}$
- Hai toán tử hiệu quả: $Find(x)$ và $Union(x,y)$.

Union-Find

- $Items = \{1, 2, 3, 4, 5, 6\}$
- $Collections = \{1, 4\}, \{3, 5, 6\}, \{2\}$
- Find(x) trả về phần tử đại diện của tập chứa x
 - ▶ Find(1) = 1
 - ▶ Find(4) = 1
 - ▶ Find(3) = 5
 - ▶ Find(5) = 5
 - ▶ Find(6) = 5
 - ▶ Find(2) = 2
- a và b thuộc cùng một tập khi và chỉ khi
Find(a) == Find(b)

Union-Find

- $Items = \{1, 2, 3, 4, 5, 6\}$
- $Collections = \{1, 4\}, \{3, 5, 6\}, \{2\}$
- $Union(x, y)$ trộn tập chứa x và tập chứa y vào nhau
 - ▶ $union(4, 2)$
 - ▶ $Collections = \{1, 2, 4\}, \{3, 5, 6\}$
 - ▶ $union(3, 6)$
 - ▶ $Collections = \{1, 2, 4\}, \{3, 5, 6\}$
 - ▶ $union(2, 6)$
 - ▶ $Collections = \{1, 2, 3, 4, 5, 6\}$

Cài đặt Union-Find

- Hợp nhất nhanh với kỹ thuật nén đường (Quick Union with path compression)
- Cực kỳ dễ cài đặt
- Cực kỳ hiệu quả

```
1 struct Union_Find {  
2     vector<int> iParent;  
3     Union_Find(int n) {  
4         iParent = vector<int>(n);  
5         for (int i = 0; i < n; ++i) {  
6             iParent[i] = i;  
7         }  
8     }  
9     // toan tu Find va Union  
10 };
```

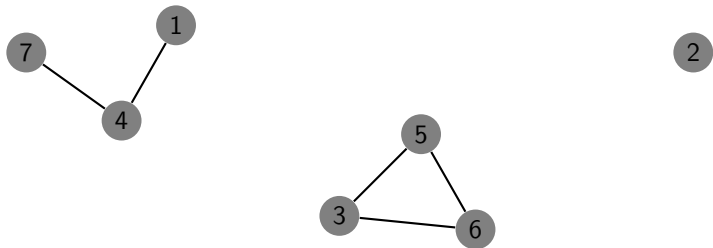
Cài đặt Union-Find

```
1 // toan tu Find va Union
2
3 int Find(int x) {
4     if (iParent[x] == x) {
5         return x;
6     } else {
7         iParent[x] = Find(iParent[x]); //Nen parent
8         return iParent[x];
9     }
10 }
11
12 void Unite(int x, int y) {
13     iParent[Find(x)] = Find(y);
14 }
```

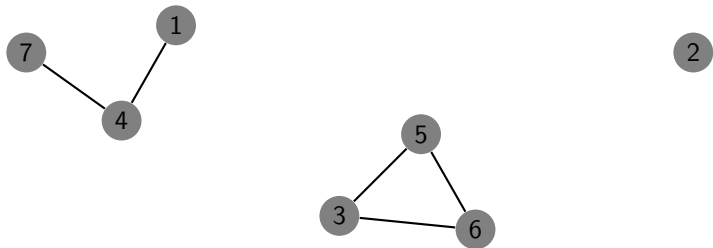
Ứng dụng của Union-Find

- Union-Find quản lý các tập không giao nhau
- Xử lý từng loại các tập không giao nhau tùy thời điểm
- Các bài toán ứng dụng quen thuộc thường trên đồ thị

Union-Find trên đồ thị

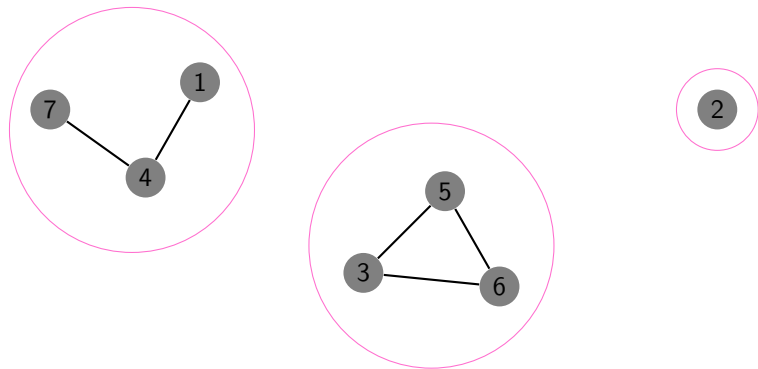


Union-Find trên đồ thị



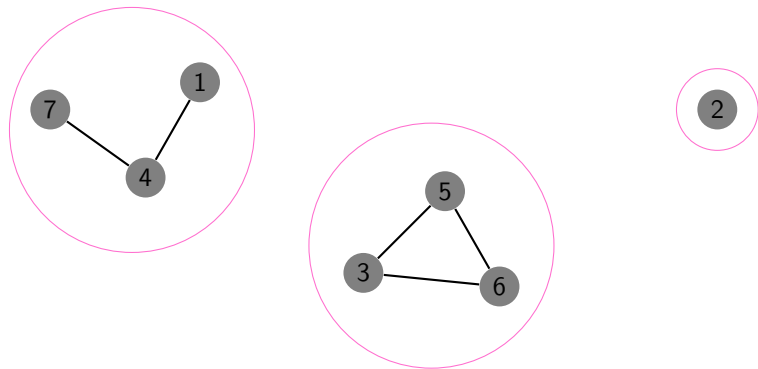
- $items = \{1, 2, 3, 4, 5, 6, 7\}$

Union-Find trên đồ thị



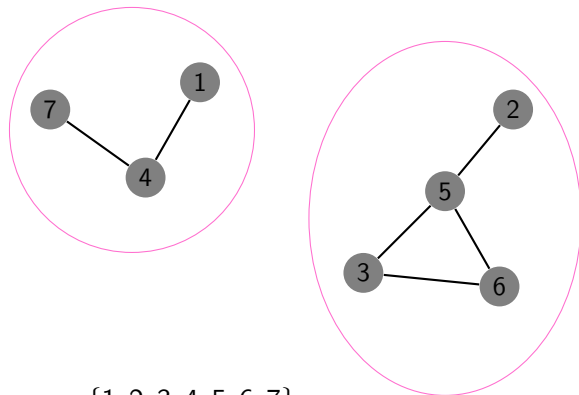
- $items = \{1, 2, 3, 4, 5, 6, 7\}$
- $collections = \{1, 4, 7\}, \{2\}, \{3, 5, 6\}$

Union-Find trên đồ thị



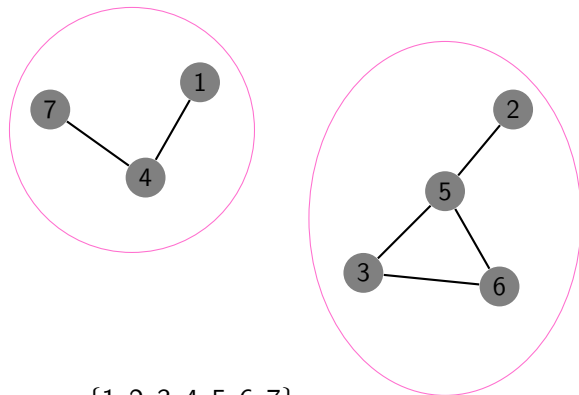
- $items = \{1, 2, 3, 4, 5, 6, 7\}$
- $collections = \{1, 4, 7\}, \{2\}, \{3, 5, 6\}$
- $union(2, 5)$

Union-find trên đồ thị



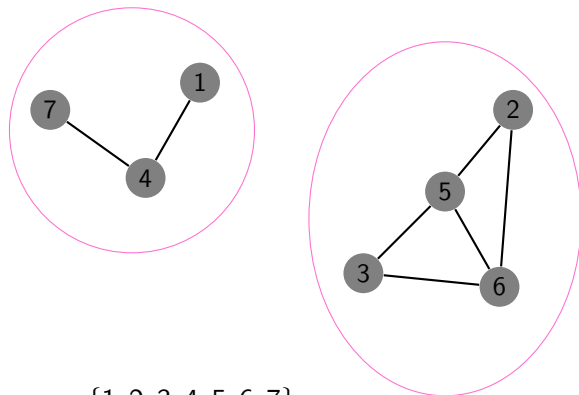
- $Items = \{1, 2, 3, 4, 5, 6, 7\}$
- $Collections = \{1, 4, 7\}, \{2, 3, 5, 6\}$

Union-find trên đồ thị



- $Items = \{1, 2, 3, 4, 5, 6, 7\}$
- $Collections = \{1, 4, 7\}, \{2, 3, 5, 6\}$
- $Union(6, 2)$

Union-Find trên đồ thị



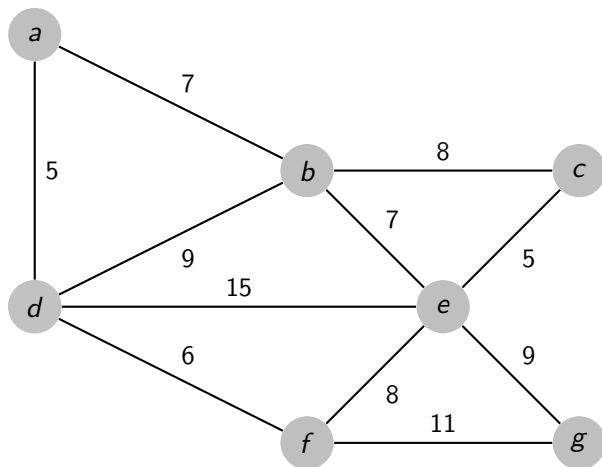
- $Items = \{1, 2, 3, 4, 5, 6, 7\}$
- $Collections = \{1, 4, 7\}, \{2, 3, 5, 6\}$

- 1 Đồ thị có trọng số
- 2 Cấu trúc dữ liệu các tập không giao nhau – UNION-FIND
- 3 Cây khung nhỏ nhất - MST**
 - Kruskal
 - Prim
- 4 Đường đi ngắn nhất
- 5 Một số bài toán kinh điển trên đồ thị
- 6 Một số đồ thị đặc biệt

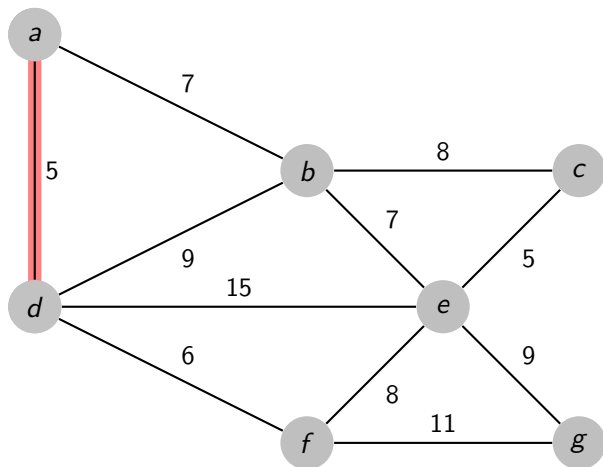
Cây khung nhỏ nhất - MST

- Cho đồ thị vô hướng có trọng số $G = (V, E)$
- $T = (V, F)$ với $F \subset E$ gọi là cây khung của G nếu như T là một cây (nghĩa là T không chứa chu trình và liên thông)
- Trọng số của T là tổng trọng số các cạnh thuộc F
- Bài toán đặt ra là tìm T có trọng số nhỏ nhất

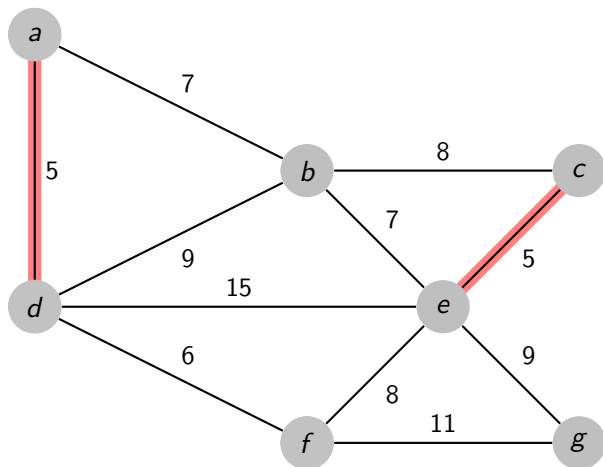
Demo thuật toán Kruskal



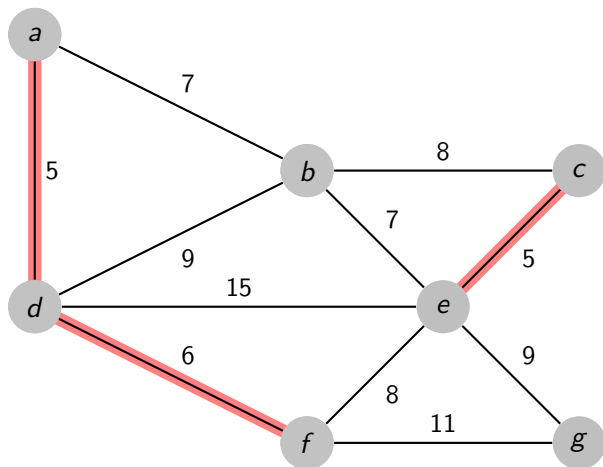
Demo thuật toán Kruskal



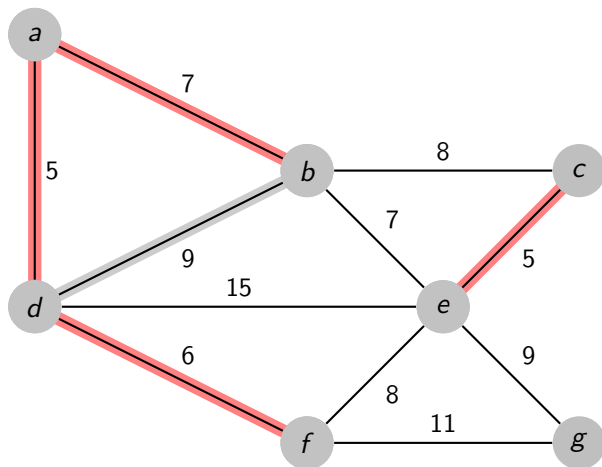
Demo thuật toán Kruskal



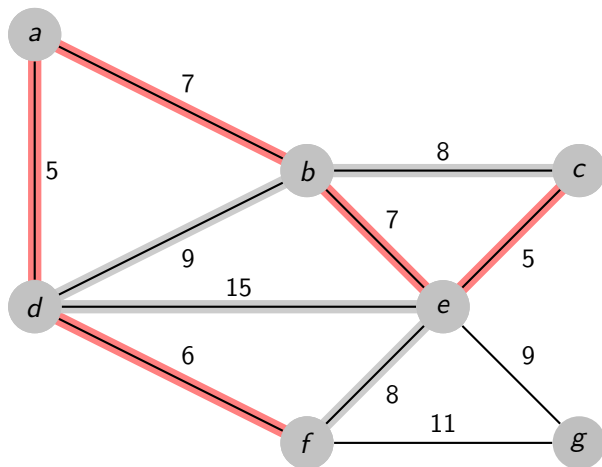
Demo thuật toán Kruskal



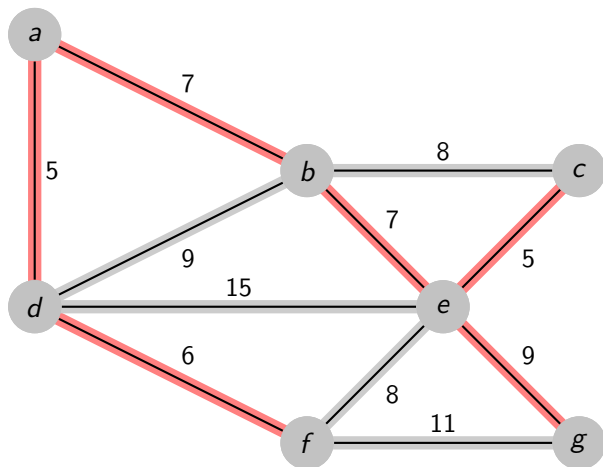
Demo thuật toán Kruskal



Demo thuật toán Kruskal



Demo thuật toán Kruskal



Thuật toán Kruskal

- Bài toán có thể giải bằng thuật toán tham lam sau
- Khởi tạo $F = \emptyset$
- Duyệt lần lượt các cạnh của đồ thị theo chiều tăng dần của trọng số
- Kết nạp vào T nếu như cạnh đó không tạo ra chu trình với các cạnh đã có trong T (có thể sử dụng Union-Find để lưu vết kiểm tra chu trình)
- Khi duyệt qua hết một lượt các cạnh ta sẽ thu được cây khung nhỏ nhất hoặc xác định không tồn tại cây khung của đồ thị
- Độ phức tạp $\mathcal{O}(|E| \log |V|)$

Hai vấn đề quan trọng khi cài đặt thuật toán Kruskal:

- 1 Làm thế nào để xét được các cạnh từ cạnh có trọng số nhỏ tới cạnh có trọng số lớn? Ta có thể thực hiện bằng cách **sắp xếp danh sách cạnh** theo thứ tự không giảm của trọng số, sau đó duyệt từ đầu đến cuối danh sách cạnh $\Rightarrow$ Có thể sử dụng thuật toán HeapSort cho phép chọn lần lượt các cạnh từ cạnh trọng số nhỏ nhất tới cạnh trọng số lớn nhất ra khỏi Heap.
- 2 Làm thế nào kiểm tra xem việc thêm một cạnh có tạo thành chu trình đơn trong T hay không? Để ý rằng các cạnh trong T ở các bước sẽ tạo thành một rừng (đồ thị không có chu trình đơn). Vì vậy muốn thêm một cạnh (u, v) vào T mà không tạo thành chu trình đơn thì (u, v) phải **nối hai cây khác nhau** của rừng T , bởi nếu u, v thuộc cùng một cây thì sẽ tạo thành chu trình đơn trong cây đó. Ban đầu, khởi tạo rừng T gồm n cây, mỗi cây chỉ gồm đúng một đỉnh $\Rightarrow$ mỗi khi xét đến cạnh nối hai cây khác nhau của rừng T thì ta sẽ kết nạp cạnh đó vào $T \Rightarrow$ hợp nhất hai cây đó lại thành một cây $\Rightarrow$ Sử dụng kỹ thuật Union-Find

Thuật toán Kruskal: Code

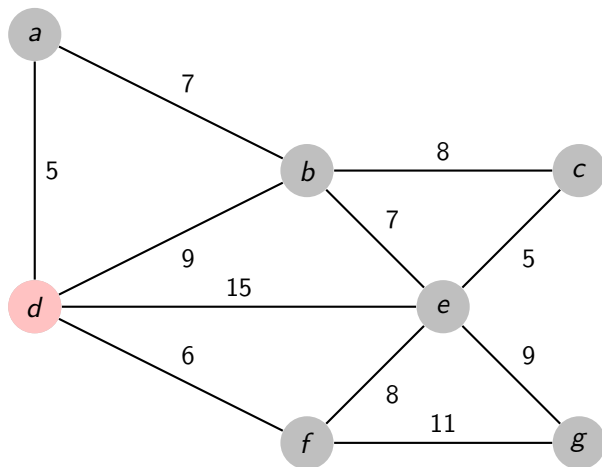
```
1  bool Edge_Cmp(const edge &a, const edge &b) {
2      return a.weight < b.weight;
3  }
4
5  vector<edge> MST(int n, vector<edge> edges) {
6      union_find UF(n);
7      sort(Edges.begin(), Edges.end(), Edge_cmp);
8      vector<edge> res;
9      for (int i = 0; i < Edges.size(); ++i) {
10         int u = Edges[i].u,
11             v = Edges[i].v;
12         if (uf.Find(u) != uf.Find(v)) {
13             UF.Unite(u, v);
14             res.push_back(Edges[i]);
15         }
16     }
17     return res;
18 }
```

- 1 Đồ thị có trọng số
- 2 Cấu trúc dữ liệu các tập không giao nhau – UNION-FIND
- 3 Cây khung nhỏ nhất - MST**
 - Kruskal
 - Prim
- 4 Đường đi ngắn nhất
- 5 Một số bài toán kinh điển trên đồ thị
- 6 Một số đồ thị đặc biệt

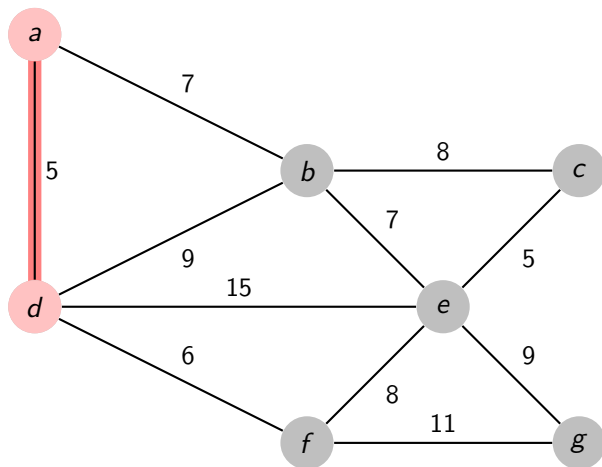
Thuật toán Prim

Thuật toán Kruskal làm việc kém hiệu quả đối với những đồ thị dày (đồ thị với số cạnh m xấp xỉ $n \times (n - 1)/2$). Trong trường hợp đó thuật toán Prim tỏ ra hiệu quả hơn. Thuật toán Prim còn gọi là phương pháp lân cận gần nhất.

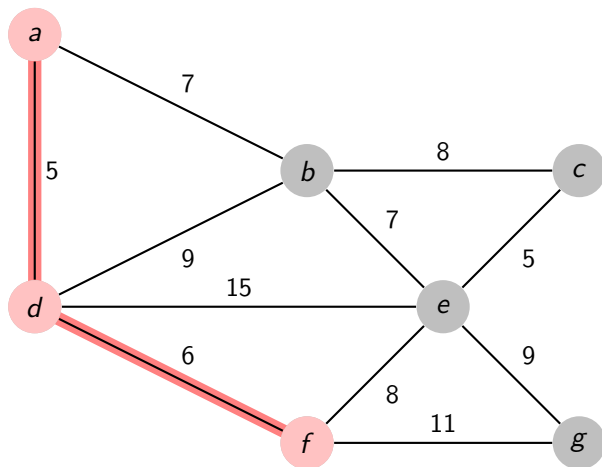
Demo thuật toán Prim



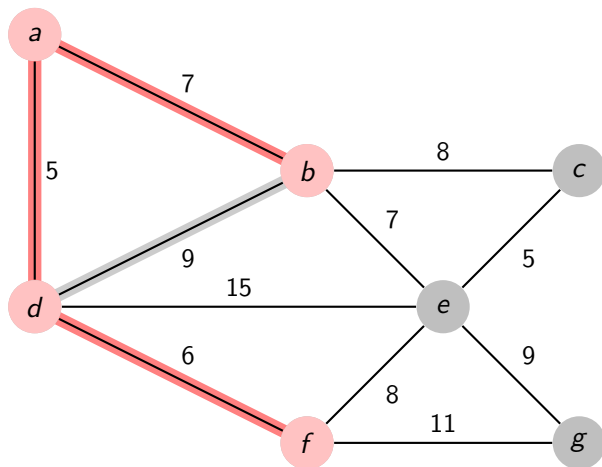
Demo thuật toán Prim



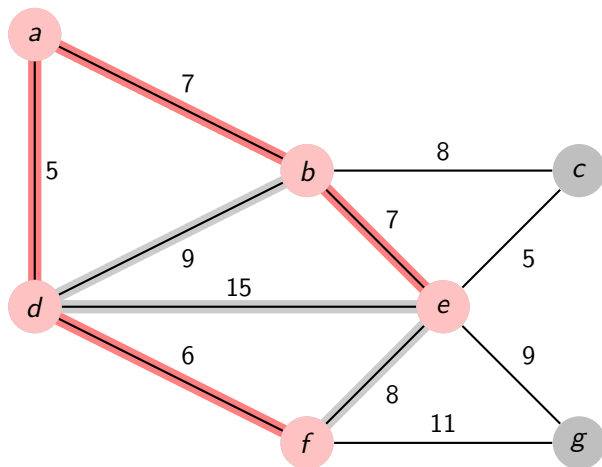
Demo thuật toán Prim



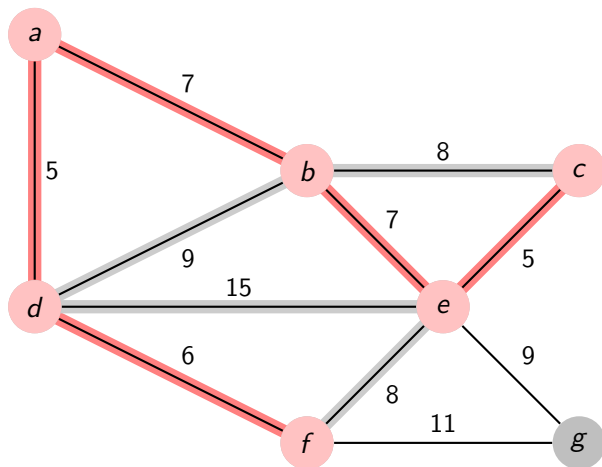
Demo thuật toán Prim



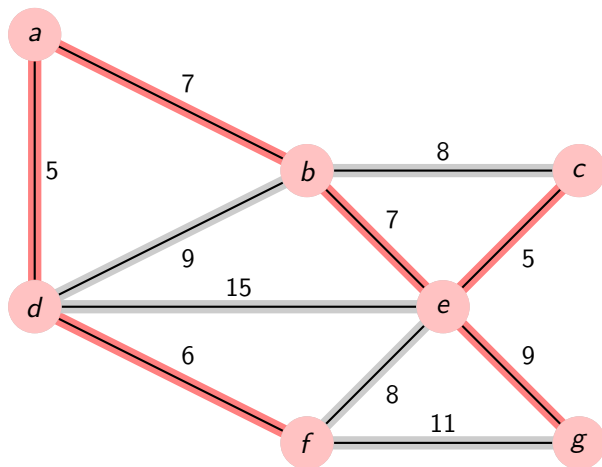
Demo thuật toán Prim



Demo thuật toán Prim



Demo thuật toán Prim



Thuật toán Prim: Mô tả

- B1: Chọn tùy ý một đỉnh s , khởi tạo $V_T = \{s\}$, $E_T = \emptyset$
- B2: Nếu $|E_T| = |V| - 1$ thì đưa ra cây $T = (V_T, E_T)$, kết thúc thuật toán
- B3: Chọn cạnh $e = (u, v, w)$ có $u \in V_T$, $v \notin V_T$ với w nhỏ nhất
- B4: Nạp đỉnh v và cạnh e vào cây: $V_T = V_T \cup \{v\}$, $E_T = E_T \cup \{e\}$. Quay lại B2
- Kết thúc thuật toán nếu chưa kết nạp được hết n đỉnh thì đồ thị đã cho không liên thông, đồ thị G không tồn tại cây khung
- Độ phức tạp $\mathcal{O}(\min(|V|^2, (|V| + |E|) \log |V|))$

Thuật toán Prim: Kỹ thuật cài đặt

- Sử dụng mảng đánh dấu bIn_T . $bIn_T[v] = false$ nếu như đỉnh v chưa được kết nạp vào T .
- Gọi $iBest_W[v]$ là khoảng cách từ v tới T . Ban đầu khởi tạo:
 - ▶ $iBest_W[1] = 0$
 - ▶ $iBest_W[2] = iBest_W[3] = \dots = iBest_W[n] = +\infty$
- Gọi $iBest_A[v]$ là đỉnh gần v nhất của cây khung
- Tại mỗi bước chọn đỉnh đưa vào T , ta sẽ chọn đỉnh u nào ngoài T và có $iBest_W[u]$ nhỏ nhất. Khi kết nạp u vào T rồi thì các nhãn $iBest_W[v]$ sẽ thay đổi nếu khoảng cách từ u đến v nhỏ hơn $iBest_W[v]$ hiện tại

Thuật toán Prim: Cài đặt $\mathcal{O}(|V|^2)$

```
1  vector<edge> Prim(int sn, vector<vector<edge>> Adj) {
2      vector<bool> bIn_T(n+1, false); //Tap_Dinh_MST
3      vector<edge> res; //Tap_Canh_MST
4      vector<int> iBest_W(n+1, 1e9), iBest_A(n+1, -1);
5      iBest_W[1] = 0;
6      while (res.size() < n-1){
7          int u = -1, v = -1, w = 1e9;
8          for (int x = 1; x <= n; ++x)
9              if (bIn_T[x] == false && iBest_W[x] < w){
10                  u = iBest_A[x], v = x, w = iBest_W[x];
11              }
12          if (v == -1) return res; //Do_Thi_Khong_Lien_Thong
13          bIn_T[v] = true;
14          for (edge e : Adj[v])
15              if (iBest_W[e.v] > e.weight){
16                  iBest_W[e.v] = e.weight;
17                  iBest_A[e.v] = e.u;
18              }
19          if (v != 1) res.push_back({u, v, w});
20      }
21      return res;
22  }
```

Thuật toán Prim: Cài đặt $\mathcal{O}((|V| + |E|) \log |V|)$

```
1  vector<edge> mst(int n, vector<vector<edge>> Adj) {
2      priority_queue< pair<int,int>, vector<pair<int,int>>,
3                      greater<pair<int,int>> > PQ;
4      vector<edge> res; //Tap_Canh_MST
5      vector<int> iBest_W(n+1, 1e9), iBest_A(n+1, -1);
6      iBest_W[1] = 0;
7      PQ.push({iBest_W[1], 1});
8      while (res.size() < n-1){
9          while(!PQ.empty() &&
10               PQ.top().first != iBest_W[PQ.top().second])
11              PQ.pop();
12          if (PQ.empty()) return res; //G_Khong_Lien_Thong
13          int w=PQ.top().first, v=PQ.top().second, u=iBest_A[v];
14          for (edge e : Adj[v])
15              if (iBest_W[e.v] > e.weight){
16                  iBest_W[e.v] = e.weight;
17                  iBest_A[e.v] = e.u;
18                  PQ.push({iBest_W[e.v], e.v});
19              }
20          if (v != 1) res.push_back({u, v, w});
21      }
22      return res;
23 }
```

- 1 Đồ thị có trọng số
- 2 Cấu trúc dữ liệu các tập không giao nhau – UNION-FIND
- 3 Cây khung nhỏ nhất - MST
- 4 Đường đi ngắn nhất
 - Dijkstra
 - Bellman-Ford
 - Floyd-Warshall
- 5 Một số bài toán kinh điển trên đồ thị
- 6 Một số đồ thị đặc biệt

Đường đi ngắn nhất: Bài toán

- Cho đồ thị có trọng số $G = (V, E)$ (vô hướng hoặc có hướng)
- Cho hai đỉnh u, v , hãy tìm đường đi ngắn nhất từ u đến v ?
- Nếu tất cả trọng số trên các cạnh đều bằng nhau, bài toán có thể giải bằng tìm kiếm theo chiều rộng BFS
- Tất nhiên là đại đa số các trường hợp không như vậy...

Đường đi ngắn nhất: Bài toán

- Có rất nhiều thuật toán hiện biết giải bài toán tìm đường đi ngắn nhất
- Cũng giống như thuật toán tìm kiếm theo chiều rộng BFS, các thuật toán này tìm đường đi ngắn nhất từ đỉnh xuất phát đến tất cả các đỉnh còn lại
- Các thuật toán phổ biến và hiệu quả bao gồm: Dijkstra, Bellman-Ford, và Floyd-Warshall

Thuật toán Dijkstra

```
1  vector<edge> Adj[100];
2  vector<int> iDist(100, INF);
3  void Dijkstra(int start) {
4      iDist[start] = 0;
5      priority_queue<pair<int, int>,
6                      vector<pair<int, int> >,
7                      greater<pair<int, int> > > PQ;
8      PQ.push(make_pair(iDist[start], start));
9      while (!PQ.empty()) {
10         int u = PQ.top().second;
11         PQ.pop();
12         for (int i = 0; i < Adj[u].size(); i++) {
13             int v = Adj[u][i].v;
14             int w = Adj[u][i].weight;
15             if (w + iDist[u] < iDist[v]) {
16                 iDist[v] = w + iDist[u];
17                 PQ.push(make_pair(iDist[v], v));
18             }
19         }
20     }
21 }
```

Thuật toán Dijkstra

- Độ phức tạp thuật toán $\mathcal{O}(|E| \log |V|)$
- Lưu ý là thuật toán chỉ đúng trong trường hợp trọng số không âm

Thuật toán Bellman-Ford

```
1 void Bellman_Ford(int n, int start) {  
2  
3     iDist[start] = 0;  
4     for (int k = 1; k < n - 1; ++k) {  
5         for (int u = 1; u <= n; ++u) {  
6             for (int j = 0; j < Adj[u].size(); ++j) {  
7                 int v = Adj[u][j].v;  
8                 int w = Adj[u][j].weight;  
9                 iDist[v] = min(iDist[v], w + iDist[u]);  
10            }  
11        }  
12    }  
13 }
```

Thuật toán Bellman-Ford

- Độ phức tạp $\mathcal{O}(|V| \times |E|)$
- Có thể sử dụng để tìm ra các chu trình trọng số âm

Thuật toán Floyd-Warshall

Sử dụng phương pháp Quy hoạch động:

- Gọi $DP(k, i, j)$ là trọng số đường đi ngắn nhất từ i đến j nếu như chỉ cho phép đi trong những đỉnh $1, \dots, k$
- Điều kiện biên 1: $DP(k, i, j) = 0$ nếu $i = j$
- Điều kiện biên 2: $DP(0, i, j) = \text{Weight}[i][j]$ nếu $(i, j) \in E$
- Điều kiện biên 3: $DP(0, i, j) = \infty$
- $$DP(k, i, j) = \min \begin{cases} DP(k-1, i, k) + DP(k-1, k, j) \\ DP(k-1, i, j) \end{cases}$$

Thuật toán Floyd-Warshall

```
1  int iDist[1001][1001];
2  int Weight[1001][1001];
3  void Floyd_Warshall(int n) {
4      for (int i = 1; i <= n; ++i) {
5          for (int j = 1; j <= n; ++j) {
6              iDist[i][j] = i == j ? 0 : Weight[i][j];
7          }
8      }
9      for (int k = 1; k <= n; ++k) {
10         for (int i = 1; i <= n; ++i) {
11             for (int j = 1; j <= n; ++j) {
12                 iDist[i][j] =
13                     min(iDist[i][j], iDist[i][k] + iDist[k][j]);
14             }
15         }
16     }
17 }
```

Thuật toán Floyd-Warshall

- Tính toàn bộ các đường đi ngắn nhất giữa các cặp đỉnh
- Độ phức tạp $\mathcal{O}(|V|^3)$
- Dễ cài đặt

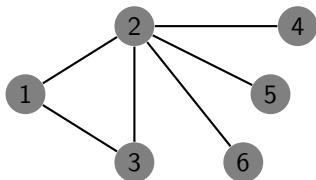
- 1 Đồ thị có trọng số
- 2 Cấu trúc dữ liệu các tập không giao nhau – UNION-FIND
- 3 Cây khung nhỏ nhất - MST
- 4 Đường đi ngắn nhất
- 5 Một số bài toán kinh điển trên đồ thị
 - Bài toán phủ đỉnh - MVC
 - Bài toán tập độc lập - MIS
 - Bài toán ghép cặp - Matching
- 6 Một số đồ thị đặc biệt

Một số bài toán kinh điển trên đồ thị

- Đây là những bài toán quan trọng và kinh điển trên đồ thị
- Thường là những bài toán khó trên đồ thị tổng quát
- Ứng dụng rộng rãi trong các bài toán thực tế khi các đồ thị tương ứng có tính chất đặc biệt
- Cũng là những bài toán cơ bản rất hay được sử dụng làm đề bài trong các kỳ thi
- Thường xuyên được ẩn chứa kín trong phát biểu bài toán

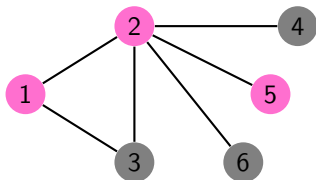
Bài toán phủ đỉnh - MVC

- Cho đồ thị vô hướng không trọng số $G = (V, E)$
- Một phủ đỉnh là một tập con các đỉnh S , sao cho với mỗi cạnh $(u, v) \in E$, hoặc u hoặc v (hoặc cả hai) thuộc S



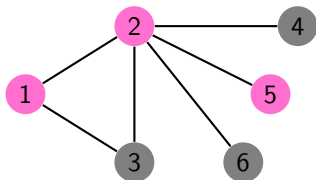
Bài toán phủ đỉnh - MVC

- Cho đồ thị vô hướng không trọng số $G = (V, E)$
- Một phủ đỉnh là một tập con các đỉnh S , sao cho với mỗi cạnh $(u, v) \in E$, hoặc u hoặc v (hoặc cả hai) thuộc S



Bài toán phủ đỉnh - MVC

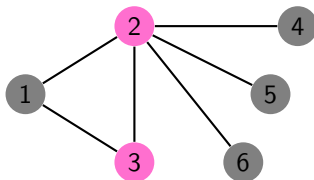
- Cho đồ thị vô hướng không trọng số $G = (V, E)$
- Một phủ đỉnh là một tập con các đỉnh S , sao cho với mỗi cạnh $(u, v) \in E$, hoặc u hoặc v (hoặc cả hai) thuộc S



- Hãy tìm một phủ đỉnh có lực lượng nhỏ nhất

Bài toán phủ đỉnh - MVC

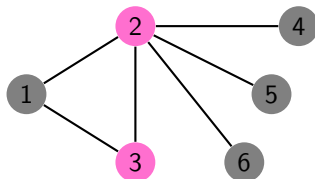
- Cho đồ thị vô hướng không trọng số $G = (V, E)$
- Một phủ đỉnh là một tập con các đỉnh S , sao cho với mỗi cạnh $(u, v) \in E$, hoặc u hoặc v (hoặc cả hai) thuộc S



- Hãy tìm một phủ đỉnh có lực lượng nhỏ nhất

Bài toán phủ đỉnh - MVC

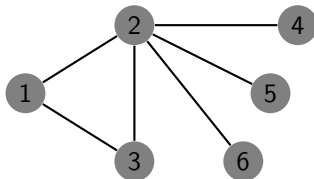
- Cho đồ thị vô hướng không trọng số $G = (V, E)$
- Một phủ đỉnh là một tập con các đỉnh S , sao cho với mỗi cạnh $(u, v) \in E$, hoặc u hoặc v (hoặc cả hai) thuộc S



- Hãy tìm một phủ đỉnh có lực lượng nhỏ nhất
- Đây là bài toán NP-khó trên đồ thị tổng quát

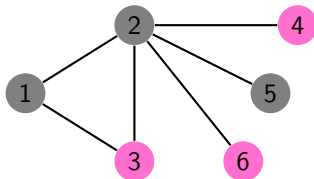
Bài toán tập độc lập - MIS

- Cho đồ thị vô hướng không trọng số $G = (V, E)$
- Một tập độc lập là một tập con các đỉnh S , sao cho không có hai đỉnh u, v nào trong S kề với nhau trong G



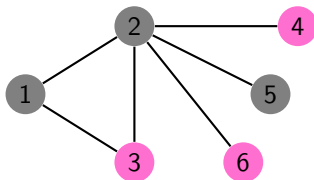
Bài toán tập độc lập - MIS

- Cho đồ thị vô hướng không trọng số $G = (V, E)$
- Một tập độc lập là một tập con các đỉnh S , sao cho không có hai đỉnh u, v nào trong S kề với nhau trong G



Bài toán tập độc lập - MIS

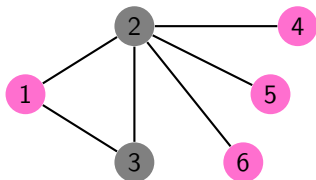
- Cho đồ thị vô hướng không trọng số $G = (V, E)$
- Một tập độc lập là một tập con các đỉnh S , sao cho không có hai đỉnh u, v nào trong S kề với nhau trong G



- Hãy tìm một tập độc lập có lực lượng lớn nhất

Bài toán tập độc lập - MIS

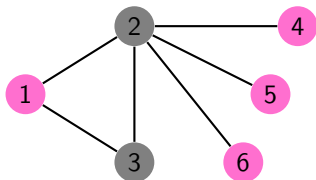
- Cho đồ thị vô hướng không trọng số $G = (V, E)$
- Một tập độc lập là một tập con các đỉnh S , sao cho không có hai đỉnh u, v nào trong S kề với nhau trong G



- Hãy tìm một tập độc lập có lực lượng lớn nhất

Bài toán tập độc lập - MIS

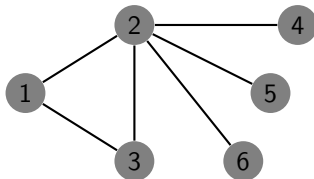
- Cho đồ thị vô hướng không trọng số $G = (V, E)$
- Một tập độc lập là một tập con các đỉnh S , sao cho không có hai đỉnh u, v nào trong S kề với nhau trong G



- Hãy tìm một tập độc lập có lực lượng lớn nhất
- Đây là bài toán NP-khó trên đồ thị tổng quát

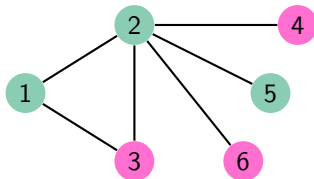
Mối quan hệ giữa MVC và MIS

- Hai bài toán trên có mối liên quan chặt chẽ với nhau
- Một tập con của các đỉnh là một phủ đỉnh khi và chỉ khi tập bù của nó là tập độc lập



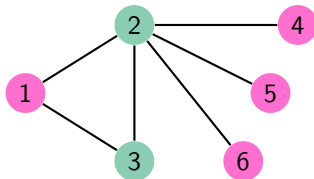
Mối quan hệ giữa MVC và MIS

- Hai bài toán trên có mối liên quan chặt chẽ với nhau
- Một tập con của các đỉnh là một phủ đỉnh khi và chỉ khi tập bù của nó là tập độc lập



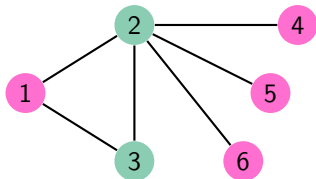
Mối quan hệ giữa MVC và MIS

- Hai bài toán trên có mối liên quan chặt chẽ với nhau
- Một tập con của các đỉnh là một phủ đỉnh khi và chỉ khi tập bù của nó là tập độc lập



Mối quan hệ giữa MVC và MIS

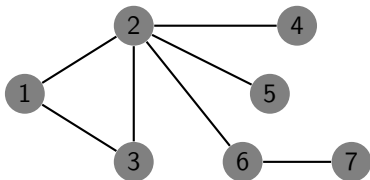
- Hai bài toán trên có mối liên quan chặt chẽ với nhau
- Một tập con của các đỉnh là một phủ đỉnh khi và chỉ khi tập bù của nó là tập độc lập



- Lực lượng của một phủ tập có kích thước nhỏ nhất cộng với lực lượng của tập độc lập có kích thước lớn nhất bằng tổng số đỉnh của đồ thị

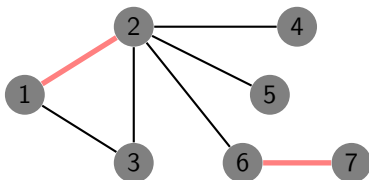
Bài toán ghép cặp

- Cho đồ thị vô hướng không trọng số $G = (V, E)$
- Một cặp ghép là một tập con các cạnh F sao cho mỗi đỉnh kề với tối đa một cạnh trong F



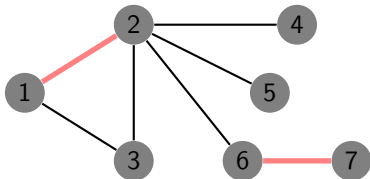
Bài toán ghép cặp

- Cho đồ thị vô hướng không trọng số $G = (V, E)$
- Một cặp ghép là một tập con các cạnh F sao cho mỗi đỉnh kề với tối đa một cạnh trong F



Bài toán ghép cặp

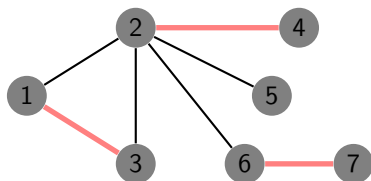
- Cho đồ thị vô hướng không trọng số $G = (V, E)$
- Một cặp ghép là một tập con các cạnh F sao cho mỗi đỉnh kề với tối đa một cạnh trong F



- Hãy tìm một cặp ghép có lực lượng lớn nhất

Bài toán ghép cặp

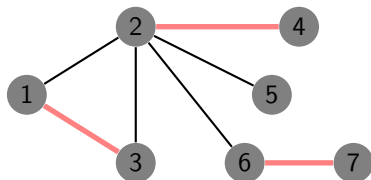
- Cho đồ thị vô hướng không trọng số $G = (V, E)$
- Một cặp ghép là một tập con các cạnh F sao cho mỗi đỉnh kề với tối đa một cạnh trong F



- Hãy tìm một cặp ghép có lực lượng lớn nhất

Bài toán ghép cặp

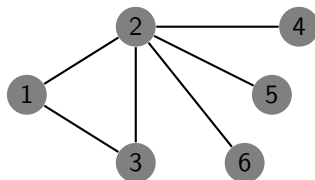
- Cho đồ thị vô hướng không trọng số $G = (V, E)$
- Một cặp ghép là một tập con các cạnh F sao cho mỗi đỉnh kề với tối đa một cạnh trong F



- Hãy tìm một cặp ghép có lực lượng lớn nhất
- Có thuật toán $\mathcal{O}(|V|^4)$ cho đồ thị tổng quát, nhưng cài đặt khá phức tạp

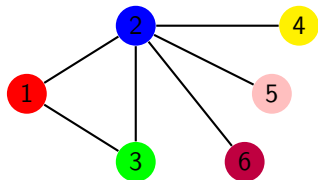
Bài toán tô màu đồ thị

- Cho đồ thị vô hướng không trọng số $G = (V, E)$
- Bài toán tô màu đồ thị yêu cầu gán các màu vào các đỉnh sao cho các đỉnh kề nhau không cùng màu



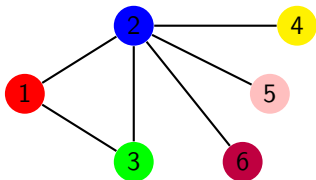
Bài toán tô màu đồ thị

- Cho đồ thị vô hướng không trọng số $G = (V, E)$
- Bài toán tô màu đồ thị yêu cầu gán các màu vào các đỉnh sao cho các đỉnh kề nhau không cùng màu



Bài toán tô màu đồ thị

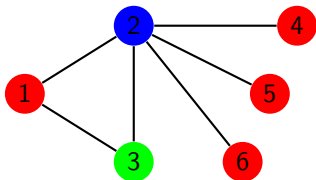
- Cho đồ thị vô hướng không trọng số $G = (V, E)$
- Bài toán tô màu đồ thị yêu cầu gán các màu vào các đỉnh sao cho các đỉnh kề nhau không cùng màu



- Hãy tìm một cách tô màu sao cho sử dụng ít màu khác nhau nhất, hay còn gọi tìm *sắc số* của đồ thị

Bài toán tô màu đồ thị

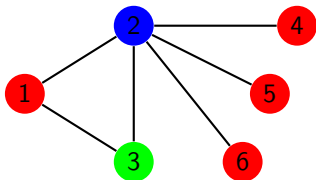
- Cho đồ thị vô hướng không trọng số $G = (V, E)$
- Bài toán tô màu đồ thị yêu cầu gán các màu vào các đỉnh sao cho các đỉnh kề nhau không cùng màu



- Hãy tìm một cách tô màu sao cho sử dụng ít màu khác nhau nhất, hay còn gọi tìm *sắc số* của đồ thị

Bài toán tô màu đồ thị

- Cho đồ thị vô hướng không trọng số $G = (V, E)$
- Bài toán tô màu đồ thị yêu cầu gán các màu vào các đỉnh sao cho các đỉnh kề nhau không cùng màu



- Hãy tìm một cách tô màu sao cho sử dụng ít màu khác nhau nhất, hay còn gọi tìm *sắc số* của đồ thị
- Đây là bài toán NP-khó trên đồ thị tổng quát

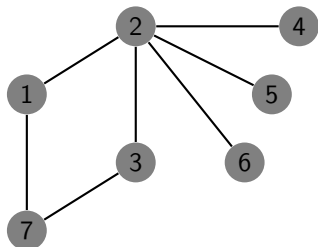
- 1 Đồ thị có trọng số
- 2 Cấu trúc dữ liệu các tập không giao nhau – UNION-FIND
- 3 Cây khung nhỏ nhất - MST
- 4 Đường đi ngắn nhất
- 5 Một số bài toán kinh điển trên đồ thị
- 6 Một số đồ thị đặc biệt
 - Đồ thị hai phía
 - Cây
 - Đồ thị có hướng không có chu trình - DAG

Một số đồ thị đặc biệt

- Hầu hết các bài toán trên đều là NP-khó trong trường hợp đồ thị tổng quát
- Còn trên một số loại đồ thị đặc biệt thì sao?
- Xét một số ví dụ

Đồ thị hai phía

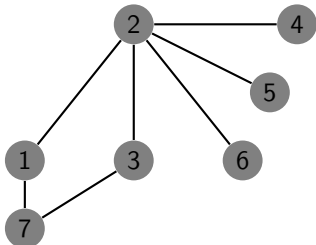
- Một đồ thị là hai phía nếu như các đỉnh được phân chia thành hai tập sao cho với mỗi cạnh (u, v) thì u và v nằm ở hai tập khác nhau



- Làm thế nào để kiểm tra một đồ thị là hai phía?

Đồ thị hai phía

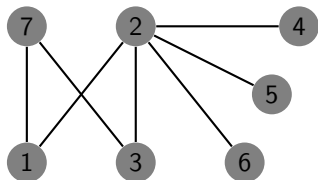
- Một đồ thị là hai phía nếu như các đỉnh được phân chia thành hai tập sao cho với mỗi cạnh (u, v) thì u và v nằm ở hai tập khác nhau



- Làm thế nào để kiểm tra một đồ thị là hai phía?

Đồ thị hai phía

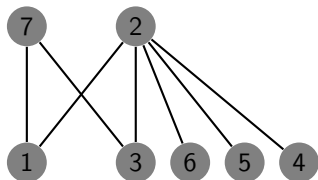
- Một đồ thị là hai phía nếu như các đỉnh được phân chia thành hai tập sao cho với mỗi cạnh (u, v) thì u và v nằm ở hai tập khác nhau



- Làm thế nào để kiểm tra một đồ thị là hai phía?

Đồ thị hai phía

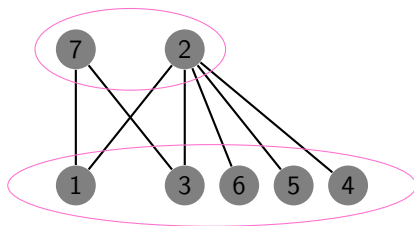
- Một đồ thị là hai phía nếu như các đỉnh được phân chia thành hai tập sao cho với mỗi cạnh (u, v) thì u và v nằm ở hai tập khác nhau



- Làm thế nào để kiểm tra một đồ thị là hai phía?

Đồ thị hai phía

- Một đồ thị là hai phía nếu như các đỉnh được phân chia thành hai tập sao cho với mỗi cạnh (u, v) thì u và v nằm ở hai tập khác nhau



- Làm thế nào để kiểm tra một đồ thị là hai phía?

Đồ thị hai phía

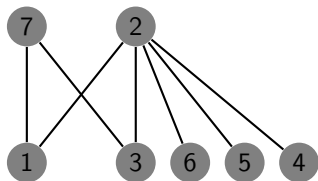
- Cần phải kiểm tra xem ta có thể chia tập đỉnh thành hai nhóm
 - Lấy một đỉnh bất kỳ, giả sử nó thuộc nhóm 1
 - Sau đó tất cả đỉnh kề với nó sẽ thuộc nhóm 2
 - Tiếp theo tất cả đỉnh kề với các đỉnh nhóm 2 đó đều phải thuộc nhóm 1
 - Và cứ kiểm tra như vậy...
-
- Ta có thể thực hiện với thuật toán tìm kiếm theo chiều sâu DFS
 - Nếu thấy điều vô lý xảy ra (nghĩa là một đỉnh phải nằm trong cả hai nhóm 1 và 2), thì đồ thị đó không phải là đồ thị hai phía

Đồ thị hai phía

```
1  vector<int> Adj[1000];
2  vector<int> iSide(1001, -1);
3  bool is_bipartite = true;
4  void Check_Bipartite(int u) {
5      for (int i = 0; i < Adj[u].size(); ++i) {
6          int v = Adj[u][i];
7          if (iSide[v] == -1) {
8              iSide[v] = 1 - iSide[u];
9              Check_Bipartite(v);
10         } else if (iSide[u] == iSide[v]) {
11             is_bipartite = false;
12         }
13     }
14 }
15 int main() {
16     for (int u = 0; u < n; u++)
17         if (iSide[u] == -1) {
18             iSide[u] = 0;
19             Check_Bipartite(u);
20         }
21     return 0;
22 }
```

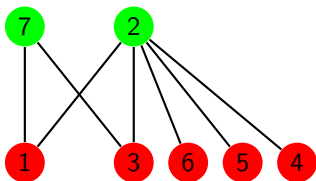
Bài toán tô màu trên đồ thị hai phía

- Làm thế nào để tô đỉnh bởi ít màu nhất trên đồ thị hai phía?



Bài toán tô màu trên đồ thị hai phía

- Làm thế nào để tô đỉnh bởi ít màu nhất trên đồ thị hai phía?



- Rất đơn giản, một phía tô bởi một màu, và phía kia tô bởi một màu khác

Bài toán ghép cặp trên đồ thị hai phía

- Bài toán ghép cặp trên đồ thị hai phía rất quen thuộc
- *xem ví dụ*
- Lưu ý là thuật toán hiệu quả tìm cặp ghép lớn nhất trên đồ thị tổng quát là phức tạp

Định lý König

- Định lý König chỉ ra rằng lực lượng của một phủ đỉnh nhỏ nhất trên đồ thị hai phía bằng với lực lượng của cặp ghép lớn nhất trên đồ thị đó
- Do đó, để tìm một phủ đỉnh nhỏ nhất trên đồ thị hai phía, ta chỉ cần tìm cặp ghép lớn nhất với thuật toán hiệu quả quen thuộc
- Và do lực lượng của tập độc lập lớn nhất chính là tổng số đỉnh của đồ thị trừ đi lực lượng của phủ đỉnh nhỏ nhất, nên ta có thể tính được tập độc lập lớn nhất một cách hiệu quả

Cây

- Cây là đồ thị vô hướng liên thông không có chu trình
- Dễ dàng kiểm tra một đồ thị là cây bằng cách kiểm tra có tồn tại cạnh ngược hay không trên cây DFS
- Một cây với n đỉnh có chính xác $n - 1$ cạnh
- Giữa mỗi cặp đỉnh u, v trên cây tồn tại duy nhất một đường đi đơn, có thể sử dụng DFS hoặc BFS để tìm đường đi này

Cây

- Các bài toán trên áp dụng trên cây thế nào?
- Tìm sắc số đỉnh trên cây?

Cây

- Các bài toán trên áp dụng trên cây thế nào?
- Tìm sắc số đỉnh trên cây?
- Thực chất cây cũng giống như đồ thị hai phía...
- Vì sao? Lấy một đỉnh bất kỳ và coi đỉnh đó là gốc của cây. Tiếp theo các đỉnh có chiều cao chẵn thì cho thuộc về một phía, còn các đỉnh chiều cao lẻ thì cho thuộc vào phía còn lại
- Vì vậy tất cả các thuật toán hiệu quả trên đồ thị hai phía đều đúng cho cây

Cây cũng rất thích hợp cho các thuật toán quy hoạch động, vì vậy rất nhiều bài toán trở nên dễ hơn nhiều trên cây vì lý do đó

Đồ thị có hướng không có chu trình - DAG

- Một đồ thị có hướng là một DAG nếu như nó không chứa chu trình
- Dễ dàng kiểm tra một đồ thị là DAG bằng cách kiểm tra xem có cạnh ngược nào không trên cây DFS
- Rất nhiều bài toán trở nên dễ dàng ở trên DAG, do có thể áp dụng quy hoạch động nhờ tính chất không có chu trình trên DAG
 - ▶ tính số lượng đường đi đơn từ u đến v
 - ▶ đường đi dài nhất từ u đến v